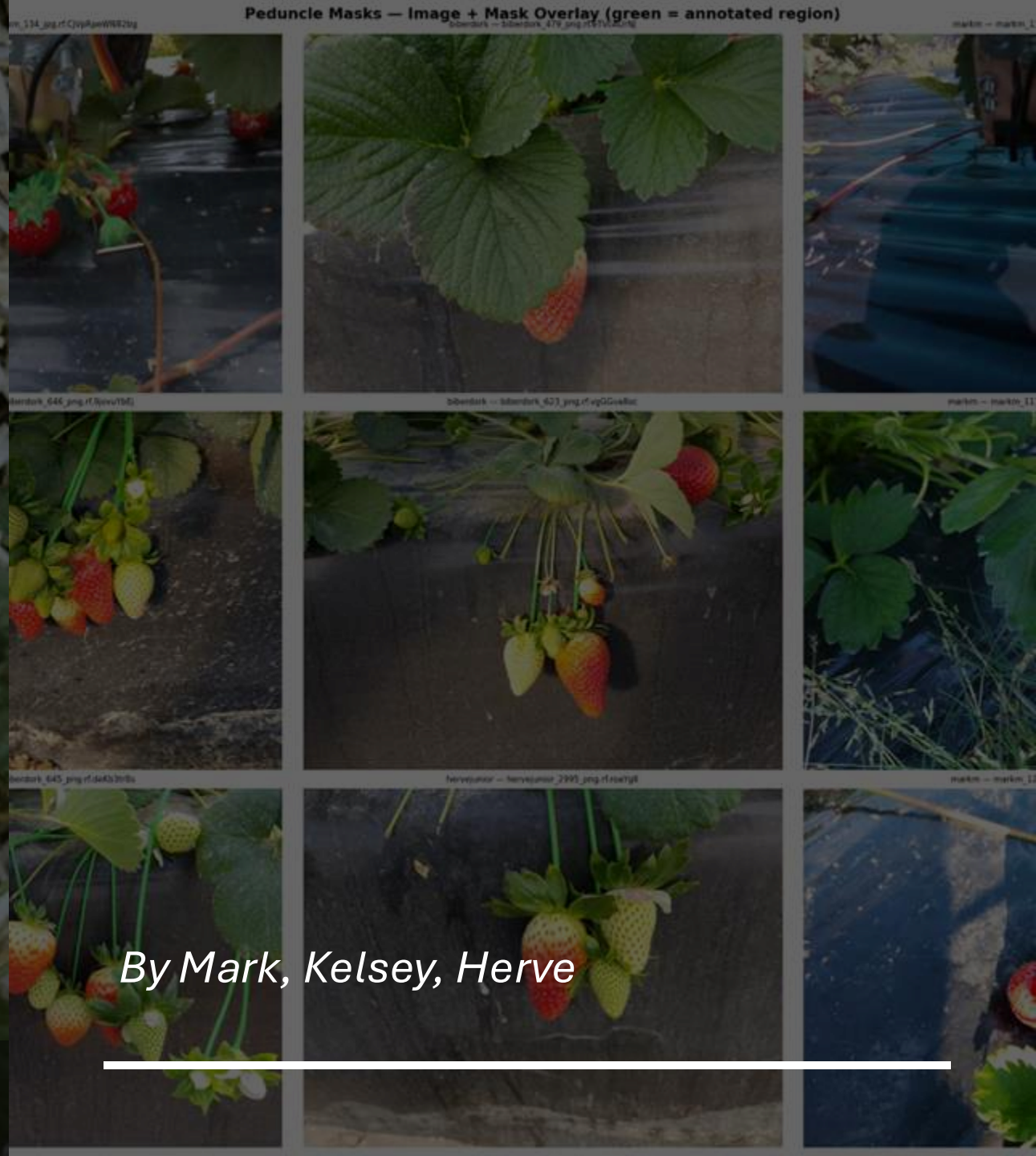
Deep Learning Pipeline for Strawberry Harvesting



By Mark, Kelsey, Herve

Course: AIDA 2158A – Neural Networks & Deep Learning

Instructor: Dr. M. Tufail

Introduction

This project presents a complete deep learning pipeline for robotic strawberry harvesting using RGB images. The system integrates multiple stages, including YOLO11s-based instance segmentation to detect strawberries, automated region-of-interest (ROI) cropping to isolate the largest ripe fruit, U-Net segmentation to extract the crown–stem–peduncle structure, and Principal Component Analysis (PCA) to estimate stem orientation for robotic grasping. The dataset consists of 3,100 annotated strawberry images and 357 manually annotated stem masks generated using SAM-assisted tools, enabling precise feeding for agricultural robotics applications.

Project Overview

End-to-end deep learning perception pipeline

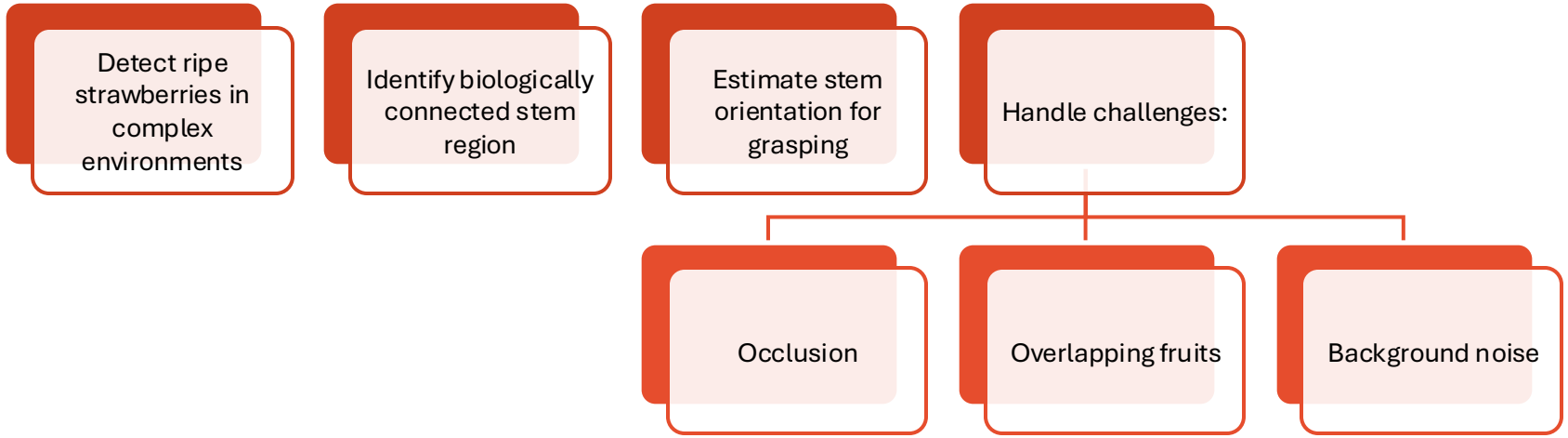
Detection: YOLOv11s -seg to identify and segment ripe strawberries.

ROI Generation: Extracting the harvest target for focused processing.

Segmentation: U-Net to isolate the crown-stem-peduncle structure.

Angle Extraction: PCA to determine the estimated robotic gripper orientation.

Problem Definition



Motivation

1

Manual harvesting is labor-intensive

2

Need for automation in agriculture

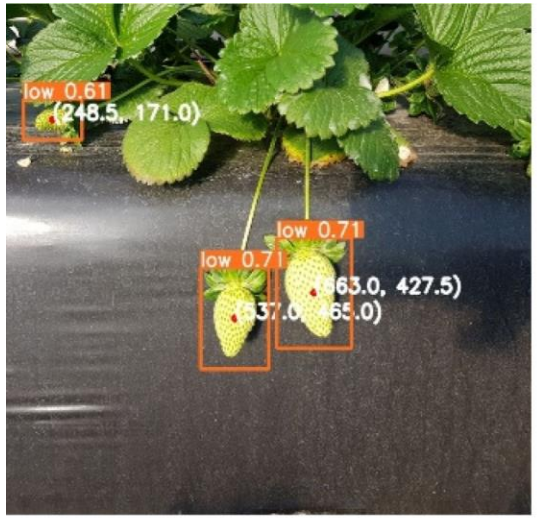
3

Robots require:

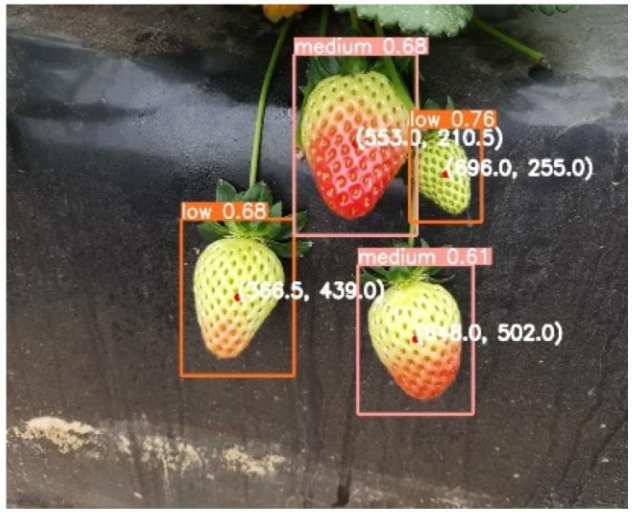
- Accurate fruit detection
- Precise stem localization
- Reliable grasp direction

Dataset

- Total: **3,100 RGB images**
- YOLO segmentation labels provided
- Split:
- Train: 2,800
- Validation: 100
- Test: 200
- Additional:
  - **357 manually annotated masks**
  - Created using SAM-assisted tools



(a) Low maturity

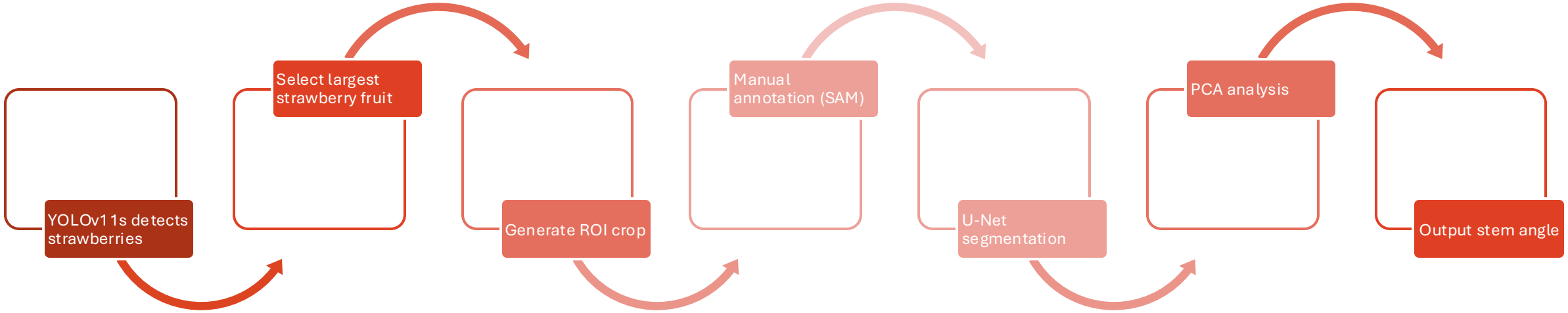


(b) Medium maturity

(c) High maturity

(d) Target is obscured

Pipeline Workflow



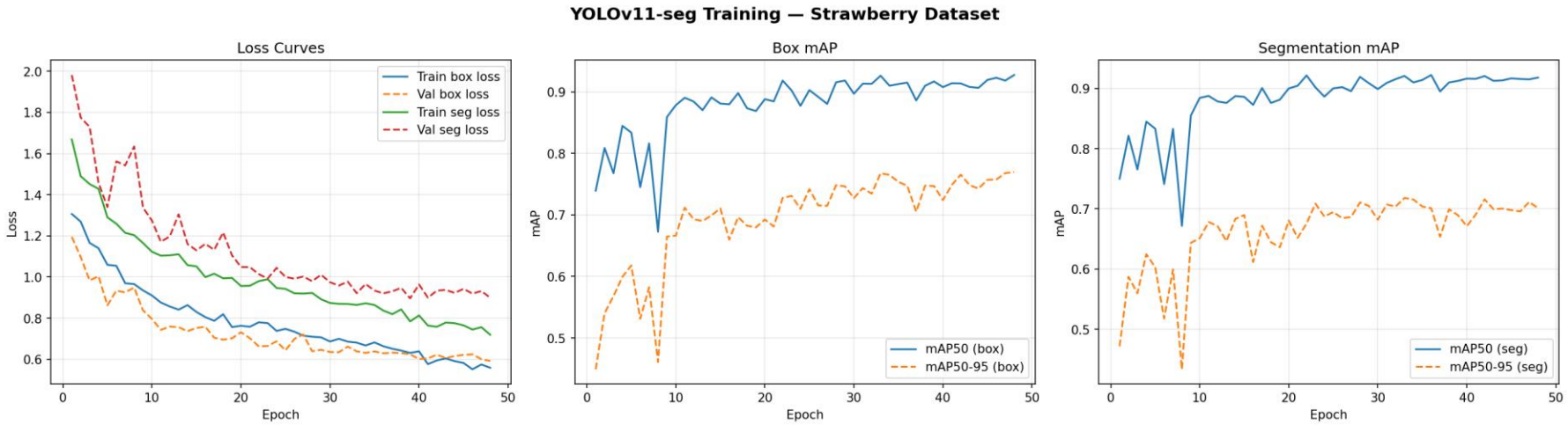
Module 1: YOLOv11 Detection

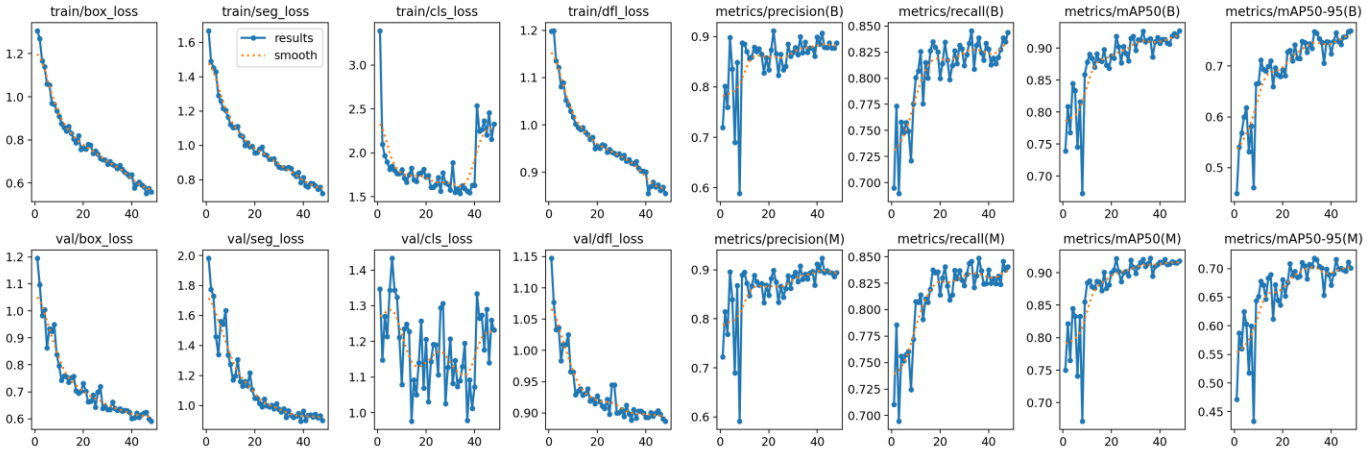
- **Task:** Detect and segment strawberries in field images.
- **Methodology:**
- Trained `yolo11s-seg` model.
- Generated automatic Region of Interest (ROI) crops based on detected bounding boxes.
- **Importance:** Focusing on the local crop allows the system to ignore background noise and concentrate on the harvest target.

```python
nb_img = np.array(Image.open(os.path.join(VAL_NB, fname)))
h, w   = nb_img.shape[:2]

lines = []
for inst_id in np.unique(nb_img):
    if inst_id == 0:
        continue   # background
    mask = (nb_img == inst_id).astype(np.uint8) * 255
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not contours:
        continue
    # Use the largest contour for this instance
    contour = max(contours, key=cv2.contourArea)
    if cv2.contourArea(contour) < 50:   # skip tiny artefacts
        continue
    # Simplify to reduce point count while preserving shape
    epsilon = 0.002 * cv2.arcLength(contour, True)
    approx  = cv2.approxPolyDP(contour, epsilon, True)
    coords  = approx.reshape(-1, 2)
    if len(coords) < 3:
        continue
    # Normalise to 0-1
    norm = []
    for x, y in coords:
        norm.extend([x / w, y / h])
    line = "0 " + " ".join(f"{v:.6f}" for v in norm)
    lines.append(line)

out_path = os.path.join(VAL_LBLS, stem + ".txt")
if lines:
    with open(out_path, 'w') as f:
```

```python
    sorted(glob.glob(os.path.join(TRAIN2_IMGS, "*.png"))) +
    sorted(glob.glob(os.path.join(VAL_IMGS,    "*.png"))) +
    sorted(glob.glob(os.path.join(TEST_IMGS,   "*.png")))
)
print(f"Total images to process: {len(all_img_paths)}")

PAD          = 20     # pixels to pad around the detected strawberry
saved        = 0
no_detection = 0

for img_path in all_img_paths:
    stem   = os.path.splitext(os.path.basename(img_path))[0]
    out_p  = os.path.join(ROI_DIR, stem + ".png")
    if os.path.exists(out_p):          # skip already done
        saved += 1
        continue

    result = model_best.predict(img_path, imgsz=640, device=0, conf=0.01, verbose=False)[0]

    if result.boxes is None or len(result.boxes) == 0:
        no_detection += 1
        continue

    img = cv2.imread(img_path)
    H, W = img.shape[:2]

    # Find largest strawberry by bounding box area
    boxes = result.boxes.xyxy.cpu().numpy()   # [N, 4] x1 y1 x2 y2
    areas = (boxes[:, 2] - boxes[:, 0]) * (boxes[:, 3] - boxes[:, 1])
    best  = int(np.argmax(areas))
    x1, y1, x2, y2 = boxes[best].astype(int)

    # Pad and clamp to image bounds
    x1 = max(0, x1 - PAD)
    y1 = max(0, y1 - PAD)
```

Training Result





The training results show a steady decrease in both training and validation losses, indicating stable learning and convergence. The model achieves strong performance, with mAP50 reaching approximately 0.93 for detection and 0.92 for segmentation, along with balanced precision and recall. Additionally, the trained model is used to generate Regions of Interest (ROI) by automatically selecting and cropping the largest detected strawberry in each image, reducing background noise, and improving focus for downstream tasks. The close alignment between training and validation curves suggests minimal overfitting, with convergence reached around epoch 45–48.

Validation results

The validation results show that the model accurately detects strawberries in various conditions, with predictions closely matching the ground truth. It performs well even in complex scenes with multiple fruits, demonstrating strong generalization. Some minor missed detections occur in occluded areas, but overall performance remains reliable.

Module 2: Data Annotation

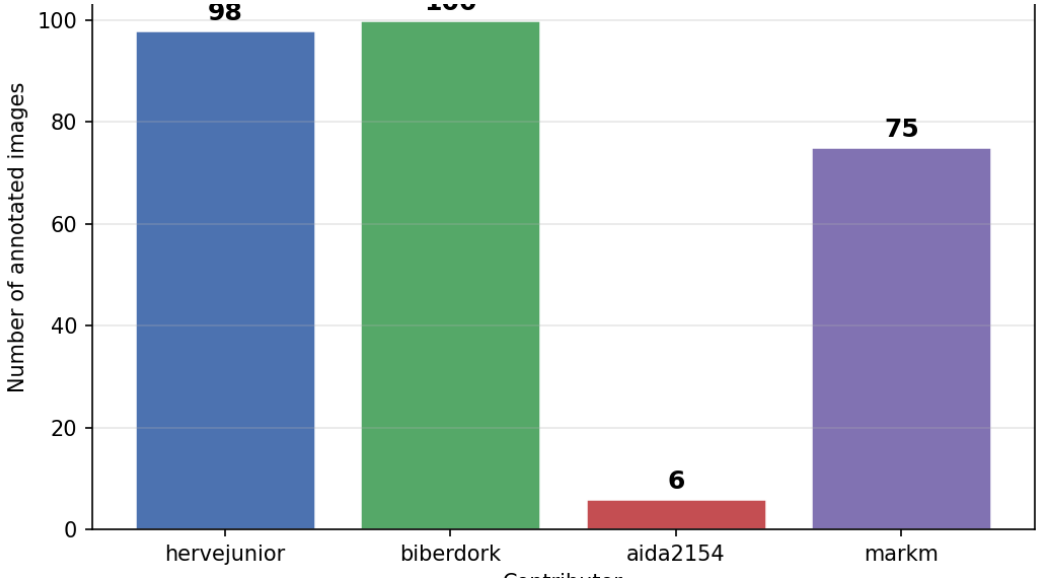
The annotation process involves manually labeling the crown–stem–peduncle region of the target strawberry using SAM-assisted tools. A total of 357 images were annotated by multiple contributors, capturing the peduncle, calyx, and upper fruit region while excluding irrelevant background. These annotations were then converted into binary masks using OpenCV, creatinga unified dataset for training the U-Net segmentation model.

```python
for contributor, img_dir, lbl_dir, cls_id in SOURCES:
    if not os.path.exists(img_dir) or not os.path.exists(lbl_di
        print(f"  SKIPPING {contributor} — folder not found")
        summary[contributor] = 0
        continue

    img_files = sorted([
        f for f in os.listdir(img_dir)
        if f.lower().endswith(('.png', '.jpg', '.jpeg'))
    ])

    saved   = 0
    skipped = 0

    for img_fname in img_files:
        stem      = os.path.splitext(img_fname)[0]
        img_path  = os.path.join(img_dir, img_fname)
        lbl_path  = os.path.join(lbl_dir, stem + ".txt")

        # Load image
        img = cv2.imread(img_path)
        if img is None:
            skipped += 1
            continue
        h, w = img.shape[:2]

        # Convert label to mask
        mask = polygon_label_to_mask(lbl_path, w, h, cls_id)
        if mask is None:
            skipped += 1
            continue

        # Use contributor prefix to avoid filename collisions a
        out_stem  = f"{contributor}_{stem}"
        out_img   = os.path.join(OUT_IMGS,  out_stem + ".png")
        out_mask  = os.path.join(OUT_MASKS, out_stem + ".png")

        cv2.imwrite(out_img,  img)
        cv2.imwrite(out_mask, mask)
        saved += 1
```

```python
def polygon_label_to_mask(label_path, img_w, img_h, pedu
    """Convert a YOLO polygon .txt file to a binary mask
    Returns a uint8 numpy array (H, W) with 255 where pe
    Returns None if no peduncle annotation exists in thi
    """
    mask = np.zeros((img_h, img_w), dtype=np.uint8)
    has_peduncle = False

    if not os.path.exists(label_path):
        return None

    with open(label_path, 'r') as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) < 7:      # need at least clas
                continue
            cls = int(parts[0])
            if cls != peduncle_class_id:
                continue            # skip strawberry bo
            coords = list(map(float, parts[1:]))
            # Pair up into (x, y) and scale to pixel coo
            pts = []
            for i in range(0, len(coords) - 1, 2):
                px = int(coords[i]     * img_w)
                py = int(coords[i + 1] * img_h)
                pts.append([px, py])
            if len(pts) < 3:
                continue
            poly = np.array(pts, dtype=np.int32).reshape
            cv2.fillPoly(mask, [poly], 255)
            has_peduncle = True

    return mask if has_peduncle else None
```

Module 2 Results

A total of 357 masks were generated, exceeding the required amount and providing a stronger dataset for training. The annotations effectively capture the stem structure, supporting model learning despite notable inconsistencies between annotators. The contribution graph highlights the distribution of annotations among contributors, demonstrating collaborative effort and dataset diversity.



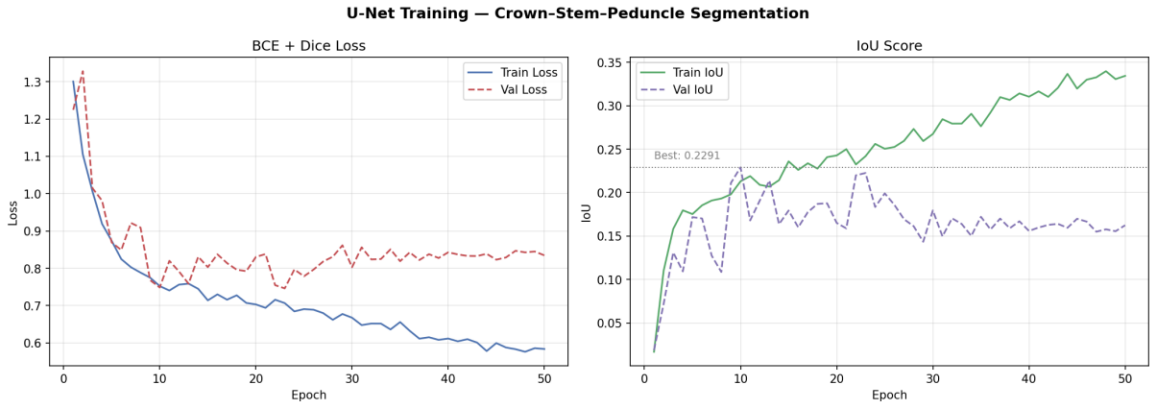
Module 3: U-Net Training

We implemented a U-Net architecture to segment the strawberry peduncle, utilizing skip connections to preserve delicate structural geometry. Trained on full images using combined BCE and Dice Loss, the model reached peak performance at epoch 10. As shown in the curves below, validation IoU peaked at 0.2291 at epoch 10; the subsequent divergence between training and validation metrics indicates the onset of overfitting after the 10th epoch, likely resulting from the limited scale and annotation variability of our dataset.

```python
def __init__(self, in_channels=3, out_channels=1, features=[64, 128, 256, 512]):
    super().__init__()
    self.encoders  = nn.ModuleList()
    self.pool      = nn.MaxPool2d(2, 2)
    self.decoders  = nn.ModuleList()
    self.upsamples = nn.ModuleList()

    # Encoder
    ch = in_channels
    for f in features:
        self.encoders.append(double_conv(ch, f))
        ch = f

    # Bottleneck
    self.bottleneck = double_conv(features[-1], features[-1] * 2)

    # Decoder
    for f in reversed(features):
        self.upsamples.append(nn.ConvTranspose2d(f * 2, f, kernel_size=2, stride=2))
        self.decoders.append(double_conv(f * 2, f))

    self.final = nn.Conv2d(features[0], out_channels, kernel_size=1)

def forward(self, x):
    skips = []
    for enc in self.encoders:
        x = enc(x)
        skips.append(x)
        x = self.pool(x)

    x = self.bottleneck(x)
    skips = skips[::-1]

    for i, (up, dec) in enumerate(zip(self.upsamples, self.decoders)):
        x = up(x)
        skip = skips[i]
        # Handle odd-dimension mismatches
        if x.shape != skip.shape:
            x = nn.functional.interpolate(x, size=skip.shape[2:])
        x = torch.cat([skip, x], dim=1)
```

```
Input (3×256×256)
  → Encoder block 1 (64)  ──────────────────────────────┐ skip
  → Encoder block 2 (128) ─────────────────────────┐ skip│
  → Encoder block 3 (256) ──────────────────┐ skip│
  → Encoder block 4 (512) ─────────────┐ skip│
  → Bottleneck      (1024)                │
  → Decoder block 4 (512) ←────────────┘
  → Decoder block 3 (256) ←──────────────────┘
  → Decoder block 2 (128) ←──────────────────────────┘
  → Decoder block 1 (64)  ←──────────────────────────────┘
  → Output conv (1×256×256) → sigmoid → binary mask
```

```python
def double_conv(in_ch, out_ch):
    """Two 3x3 convolutions each followed by BatchNorm + ReLU."""
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
    )
```



Module 3: Results

```python
samples = random.sample(list(range(len(val_ds))), min(6, len(val_ds)))

fig, axes = plt.subplots(len(samples), 4, figsize=(16, len(samples) * 4))
fig.suptitle("U-Net Predictions — Val Set", fontsize=13, fontweight='bold')
col_titles = ["Input Image", "Ground Truth", "Prediction", "Overlay"]
for ax, title in zip(axes[0], col_titles):
    ax.set_title(title, fontsize=11, fontweight='bold')

for row, idx in enumerate(samples):
    img_t, mask_t = val_ds[idx]
    with torch.no_grad():
        pred = model(img_t.unsqueeze(0).to(DEVICE)).squeeze().cpu()

    # Denormalise image
    img_disp = (img_t * std + mean).clamp(0,1).permute(1,2,0).numpy()
    gt_disp  = mask_t.squeeze().numpy()
    pr_disp  = (pred > 0.5).float().numpy()

    # Overlay
    overlay = img_disp.copy()
    overlay[pr_disp == 1] = overlay[pr_disp == 1] * 0.5 + np.array([0, 0.8, 0.3]) * 0.5

    iou = iou_score(pred.unsqueeze(0), mask_t).item()

    axes[row][0].imshow(img_disp)
    axes[row][1].imshow(gt_disp,  cmap='gray')
    axes[row][2].imshow(pr_disp,  cmap='gray')
    axes[row][3].imshow(overlay)
    axes[row][3].set_ylabel(f"IoU: {iou:.3f}", fontsize=9)

    for ax in axes[row]:
        ax.axis('off')
```

The U-Net learned to identify the general stem region, achieving a best validation IoU of 0.2291 at epoch 10. After epoch 10, training IoU continued rising while validation IoU declined — a clear sign of overfitting driven by three factors: a small dataset (299 training images), inconsistent annotation styles across four contributors, and training on full images where the stem occupies only 2–5% of the frame.

Module 3: Problems and fixes

- **Problem 1:** We had under 300 images, we should have had over 1000
  - Fix: More images
- **Problem 2:** 4 people provided annotation, so it was inconsistent
  - Fix: One person does it all
- **Problem 3:** Tiny stems in a big image make them hard to find
  - Fix: Crop them!
- **Problem 4:** Peaked early, best result after epoch 10
  - Fix: Stop training earlier. We can do this by setting patience=5 instead of patience=15

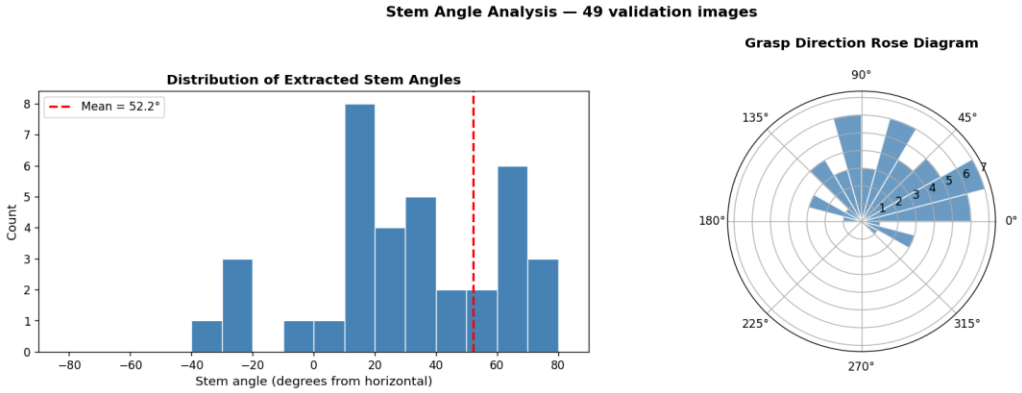
Despite this, the predicted masks were sufficient to feed the downstream PCA module, which successfully extracted stem orientation angles from 50 of 57 validation images. The pipeline works end-to-end. With ROI-crop training, consistent annotation guidelines, and 500+ labelled examples, we would expect validation IoU to reach 0.40–0.50.

Module 4: PCA Stem Orientation

PCA is applied to U-Net predicted masks to estimate stem orientation. The masks are binarized using a 0.3 threshold and refined by keeping only the largest connected component to remove noise. PCA then extracts the principal axis from foreground pixels, providing an estimated stem angle for robotic grasping.

```python
    fit 2-component PCA, return principal axis angle in degrees.
    Returns None if fewer than 10 foreground pixels.
    """
    ys, xs = np.where(mask > 127)
    if len(xs) < 10:
        return None
    coords = np.column_stack([xs, ys]).astype(float)
    pca = PCA(n_components=2)
    pca.fit(coords)
    # Principal component direction vector
    dx, dy = pca.components_[0]
    # Angle from horizontal, in degrees (-90 to +90)
    angle = np.degrees(np.arctan2(dy, dx))
    return angle

results = []   # list of dicts
for img_path in val_imgs:
    mask_pred = predict_mask(img_path)
    angle     = pca_angle(mask_pred)
    gt_path   = MASK_DIR / img_path.name
    results.append({
        'name'      : img_path.name,
```

```python
# Cell 4 — Run U-Net inference + PCA on each val image

tf = transforms.Compose([
    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225]),
])

THRESHOLD = 0.3   # v2: lowered from 0.5 — recovers uncertain stem pixels

def predict_mask(img_path):
    """Return binarised 256x256 predicted mask (numpy uint8), largest component only."""
    img = Image.open(img_path).convert('RGB')
    tensor = tf(img).unsqueeze(0).to(DEVICE)
    with torch.no_grad():
        prob = model(tensor)  # sigmoid already applied inside model.forward()
    prob = prob.squeeze().cpu().numpy()
    mask = (prob > THRESHOLD).astype(np.uint8) * 255
    # v2: keep only the largest connected component — removes isolated noise blobs
    num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(mask)
    if num_labels > 1:
        largest = 1 + np.argmax(stats[1:, cv2.CC_STAT_AREA])
        mask = np.where(labels == largest, 255, 0).astype(np.uint8)
    return mask
```



Module 4: Results

The final PCA-based module effectively translates U-Net segmentation masks into actionable geometric data. As shown in the validation gallery, the extracted principal axis aligns with the general stem direction in the majority of validation images, providing a usable vector for robotic gripper orientation. With a mean stem angle of 59.2°, this module demonstrates that a robust post-processing workflow; combining probability thresholding with largest-component filtering and successfully extracts approximate harvest angles despite the inherent complexity of field imagery.

Summary and Limitations

The pipeline achieved strong results in detection and ROI generation — YOLOv11 reached mAP50 of 0.927 and generated ROI crops for all 3,100 images. Four contributors produced 357 annotated masks, exceeding the 100-image requirement. The U-Net reached a best validation IoU of 0.23 before overfitting, limited by dataset size, annotation inconsistency, and full-image training. PCA extracted stem angles from 50 of 57 validation images, with a mean of 59°. The pipeline is a functional proof of concept; production deployment would require more data and consistent annotations.

🚀 Future Work

- Increase dataset size to improve model generalization
- Standardize annotation guidelines to reduce inconsistencies
- Train U-Net on ROI crops to minimize background noise
- Apply data augmentation techniques for better robustness
- Use pretrained encoder backbones to enhance segmentation performance

Conclusion

This project successfully developed an end-to-end deep learning pipeline for strawberry harvesting by integrating detection, segmentation, and geometric analysis. The system achieved strong detection performance and demonstrated the ability to estimate stem orientation for robotic grasping. Overall, the pipeline provides a proof-of-concept pipeline demonstrating the feasibility of deep learning for agricultural automation.